

Diseño y Análisis de Algoritmos

Ayudantía 4: Técnicas de Diseño de Algoritmos

Ayte. Cristian Nettle

Pauta

1. Count Zeroes

Usando búsqueda binaria: Dado que el string es de la forma $1^m 0^n$, se puede usar búsqueda binaria para encontrar el primer cero. Sea i la posición del primer cero; entonces el número de ceros es $|S| - i$.

COUNT-ZEROES(S)

```
1  low = 0, high = |S| - 1
2  while low ≤ high
3      mid = ⌊(low + high)/2⌋
4      if S[mid] = 1
5          low = mid + 1
6      else
7          high = mid - 1
8  return |S| - low
```

2. Sorting with Few Inversions

Usando Insertion Sort: Si el número de inversiones es pequeño ($k \ll N$), Insertion Sort es eficiente, ya que su complejidad es $O(N + k)$. Merge Sort tiene complejidad $O(N \log N)$ y no aprovecha el bajo número de inversiones. Selection Sort siempre realiza $O(N^2)$ comparaciones.

3. Interval Scheduling

Usando algoritmo Greedy: Ordenar los intervalos por tiempo de término creciente y seleccionar iterativamente cada intervalo compatible con los ya elegidos.

INTERVAL-SCHEDULING(R)

```
1  Ordenar  $R$  por  $f(i)$  creciente
2   $A = \{\}$ 
3   $last\_end = -\infty$ 
4  for cada  $i$  en  $R$ 
5      if  $s(i) \geq last\_end$ 
6          Agregar  $i$  a  $A$ 
7           $last\_end = f(i)$ 
8  return  $A$ 
```